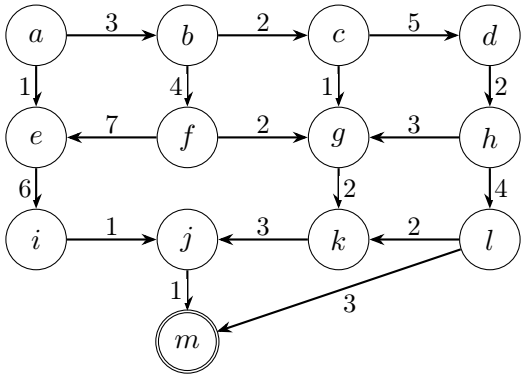
Artificiële intelligentie

Theoretisch practicum 1: Search en CSP

Vraag 1: Zoekalgoritmes

Beschouw volgende gewogen graaf. We zoeken een pad van a naar m. Geef aan welk pad wordt gevonden door onderstaande algoritmes. Uiteraard gebruiken enkel de algoritmes voor gewogen grafen de gewichten. In het geval dat er keuze is welke node eerst gekozen wordt, zal steeds de alfabetische volgorde gevolgd worden. Bijvoorbeeld: we starten in a en daarna gaan we verder met 1 van de kinderen van a. Dit zijn b en e. Omdat b voor e komt, kiezen we eerst b.



(a) Breadth-first tree search

(b) Depth-first tree search

(c) Uniform Cost tree search

We maken gebruik van tree search wat wil zeggen dat we niet bijhouden of een node al eerder via een ander pad werd gegenereerd.

Vraag 2: Admissibiliteit en consistentie

Gebruik opnieuw de graaf uit vraag 1, met doelnode m. De drie heuristieken zijn:

	a	b	c	d	e	f	g	h	i	j	k	l	m
h_1	0	0	0	0	0	0	0	0	0	0	0	0	0
h_2	8	8	4	7	7	6	5	5	1	1	3	3	0
h_3	9	9	7	9	8	8	6	7	2	1	4	3	0

(a) Welke van deze drie heuristieken zijn admissible, welke van deze drie heuristieken zijn consistent? Indien een heuristiek niet voldoet, geef dan een minimal set van nodes die dit aantonen.

(b) Bewijs of geef een tegenvoorbeeld: *elke admissibele heuristiek is consistent.* Je mag de graaf en heuristieken hierboven gebruiken.

(c) Bewijs of geef een tegenvoorbeeld: *elke consistente heuristiek is admissibel.*

(d) Bewijs of geef een tegenvoorbeeld: Als de heuristiek die A* gebruikt, admissiebel is, dan is A* optimaal. (Optimaal betekent hier dat A* steeds een kortste pad vindt als er een pad naar een doel-node bestaat).

Vraag 3: Minimax en alpha-beta pruning

Beschouw volgende positie in het spel tic-tac-toe. O is aan de beurt:

	O	X
	X	
O	X	

Gebruik alpha-beta pruning om de volgende zet voor O te bepalen. Laat duidelijk zien welke delen van de spelboom al dan niet geëxploreerd worden, en geef aan waarom bepaalde nodes niet worden bekeken. Je mag zelf de volgorde bepalen waarin je de kinderen van een node evalueert, maar zorg ervoor dat in jouw voorbeeld minstens 1 node gesnoeid kan worden.

Vraag 4: Een CSP modelleren en oplossen met backtracking

Op een badmintontornooi spelen vier spelers S, A, J en V alle zes mogelijke enkelwedstrijden: SA, SJ, SV, AJ, AV en JV. Er zijn twee velden: center court A en side court B. Een wedstrijd duurt precies één uur en kan starten om 9, 10, 11 of 12 uur. Het tornooi eindigt om 13 uur.

- Een speler kan niet aan twee wedstrijden tegelijk deelnemen.
- Op een veld kan slechts één wedstrijd tegelijk plaatsvinden.
- Alle wedstrijden van speler S moeten op center court A plaatsvinden.

Een waarde zoals $10B$ betekent: de wedstrijd begint om 10 uur op veld B. De letter A kan dus een speler of een veld aanduiden; de context bepaalt welke betekenis bedoeld wordt.

(a) Modelleer het probleem als een CSP. Geef de variabelen, hun betekenis en hun domeinen. Formuleer alle constraints precies. Maak onderscheid tussen het verbod op een gedeeld tijdstip voor wedstrijden met een gemeenschappelijke speler en het verbod op een gedeelde combinatie van tijdstip en veld.

(b) Los het CSP op met gewone backtracking, met de volgende **vaste** volgordes:

variabelen: $SA,\ AJ,\ AV,\ SJ,\ SV,\ JV$;

waarden: $9A,\ 9B,\ 10A,\ 10B,\ 11A,\ 11B,\ 12A,\ 12B$.

Sla waarden over die niet in het domein van de gekozen variabele zitten. Controleer elke kandidaat tegen de reeds toegewezen variabelen. Gebruik geen forward checking, arc consistency of dynamische ordening. Noteer elke aanvaarde toekenning, elke backtrack en de reden voor een mislukte tak. Ga door tot de eerste volledige oplossing. Vul vervolgens het schema hieronder in.

Startuur	Veld A	Veld B
9 uur		
10 uur		
11 uur		
12 uur		

Extra vragen

Extra 1: Heuristieken combineren

Laat h_a en h_b twee niet-negatieve admissibele heuristieken zijn voor hetzelfde zoekprobleem. Beide zijn nul in de doeltoestanden.

(a) Bewijs dat $h_{\max}(v) = \max\{h_a(v), h_b(v)\}$ admissibel is.

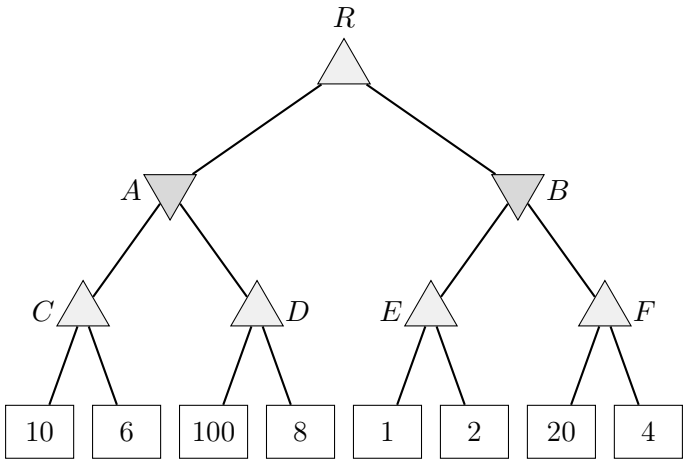
(b) Is $h_{\text{som}}(v) = h_a(v) + h_b(v)$ altijd admissibel? Bewijs je antwoord of geef een tegenvoorbeeld met concrete kosten en heuristiekwaarden.

(c) Veronderstel nu dat h_a en h_b ook consistent zijn. Bewijs of weerleg dat $h_{\max}$ consistent is.

(d) Wat betekent het dat een heuristiek een andere *domineert*? Waarom kan een nauwkeurigere heuristiek toch tot een langere totale rekentijd leiden?

Extra 2: Toeval en beperkte zoekdiepte

Gebruik volgende boom.



Vervang de MIN-nodes A en B door kansnodes. In A wordt C gekozen met kans $\frac{3}{4}$ en D met kans $\frac{1}{4}$. In B wordt E gekozen met kans $\frac{1}{4}$ en F met kans $\frac{3}{4}$. De overige interne nodes blijven MAX-nodes.

(a) Bereken met expectimax alle interne waardes en bepaal de beste eerste zet. Vergelijk je resultaat met minimax.

(b) Mag je bij deze kansnodes dezelfde snoeiregels gebruiken als bij MIN-nodes? Leg uit waarom één reeds bekeken kind in het algemeen niet volstaat om het gemiddelde van alle kinderen te begrenzen zoals een minimum of maximum.

(c) In een realistisch spel is de boom vaak te groot om alle eindposities te bereiken. Leg uit hoe een zoekdieptelimiet en een evaluatiefunctie worden gebruikt. Blijft de garantie op een optimale zet behouden bij een willekeurige evaluatiefunctie?

Extra 3: Torens en lopers als CSP

Plaats twee torens en twee lopers op een 4×4-bord, zodat geen enkel stuk een ander stuk kan slaan. Geen twee stukken mogen hetzelfde vak innemen. Een toren bedreigt stukken op dezelfde rij of kolom; een loper bedreigt stukken op dezelfde diagonaal. Noem de kolommen a, b, c, d en de rijen $1, 2, 3, 4$.

(a) Modelleer het probleem met één variabele per stuk: T_1, T_2, L_1, L_2. Geef de domeinen en alle binaire constraints. Let op: het is voldoende dat één van twee stukken het andere kan slaan om een paar posities ongeldig te maken.

(b) Fixeer $T_1 = a1$ en $L_1 = d2$. Zoek met gewone backtracking of dit tot een volledige oplossing kan worden aangevuld. Kies eerst T_2, daarna L_2, en probeer posities in de volgorde $a1, b1, c1, d1, a2, \ldots, d4$. Noteer welke keuzes consistent zijn met de reeds toegewezen stukken en toon waarom een tak slaagt of mislukt. Geef een oplossing of toon aan dat geen oplossing bestaat voor deze gefixeerde startpositie.

Veel succes!